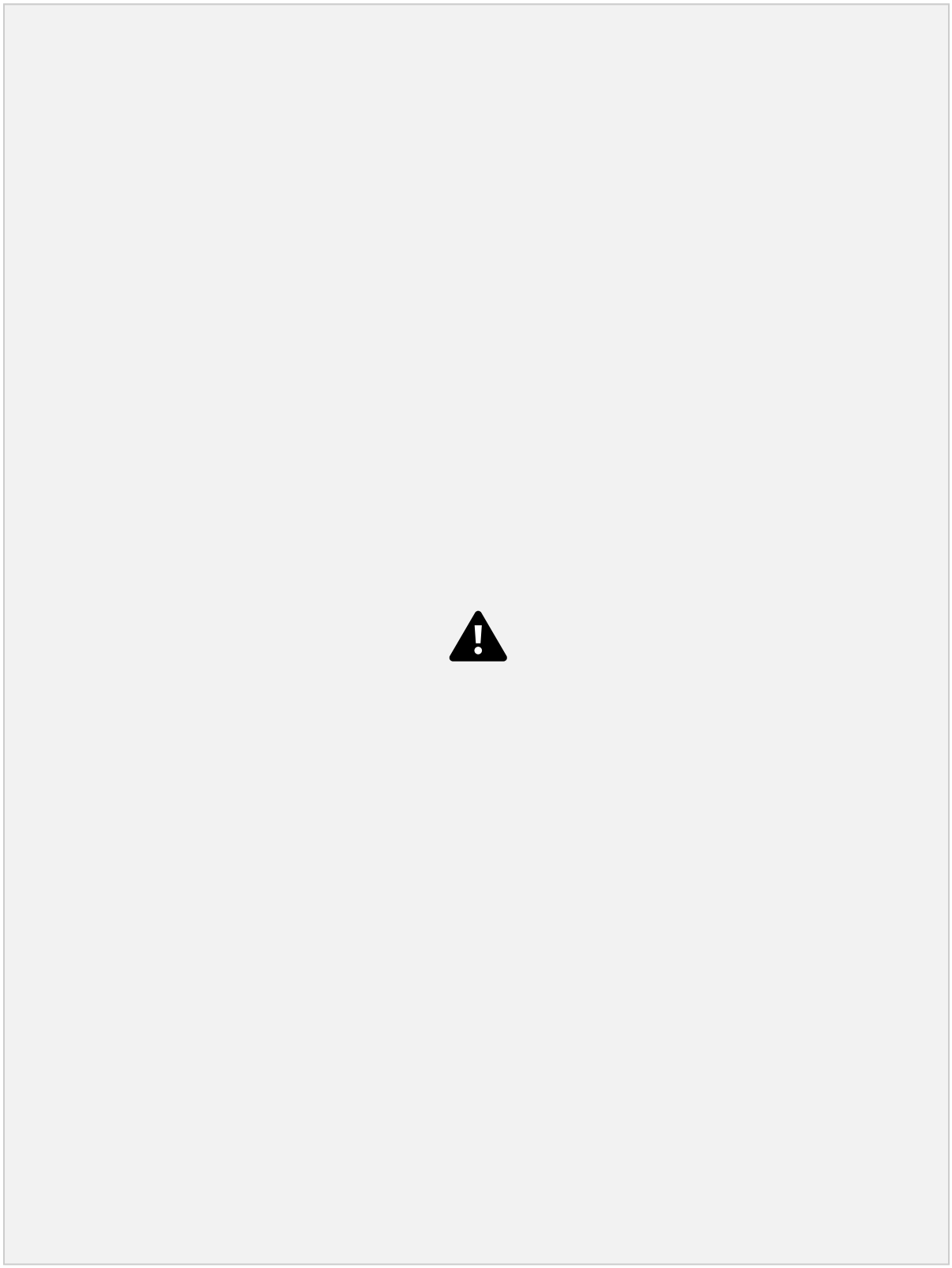
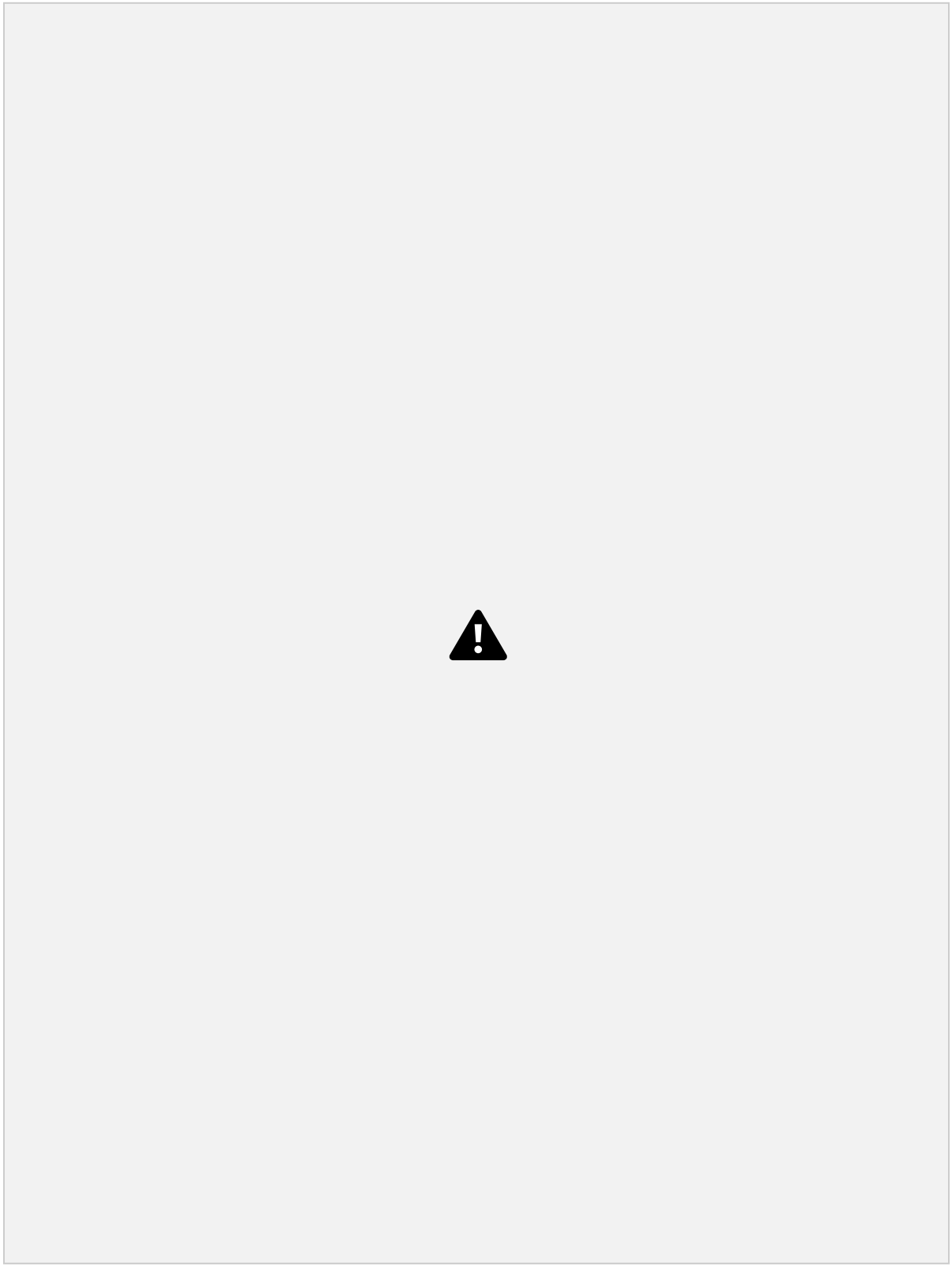
Now there is a different understanding about the system identification and digital twin part Can you go through these notes and explain again? Anomaly the process anomaly like tow pressure they meant in the document

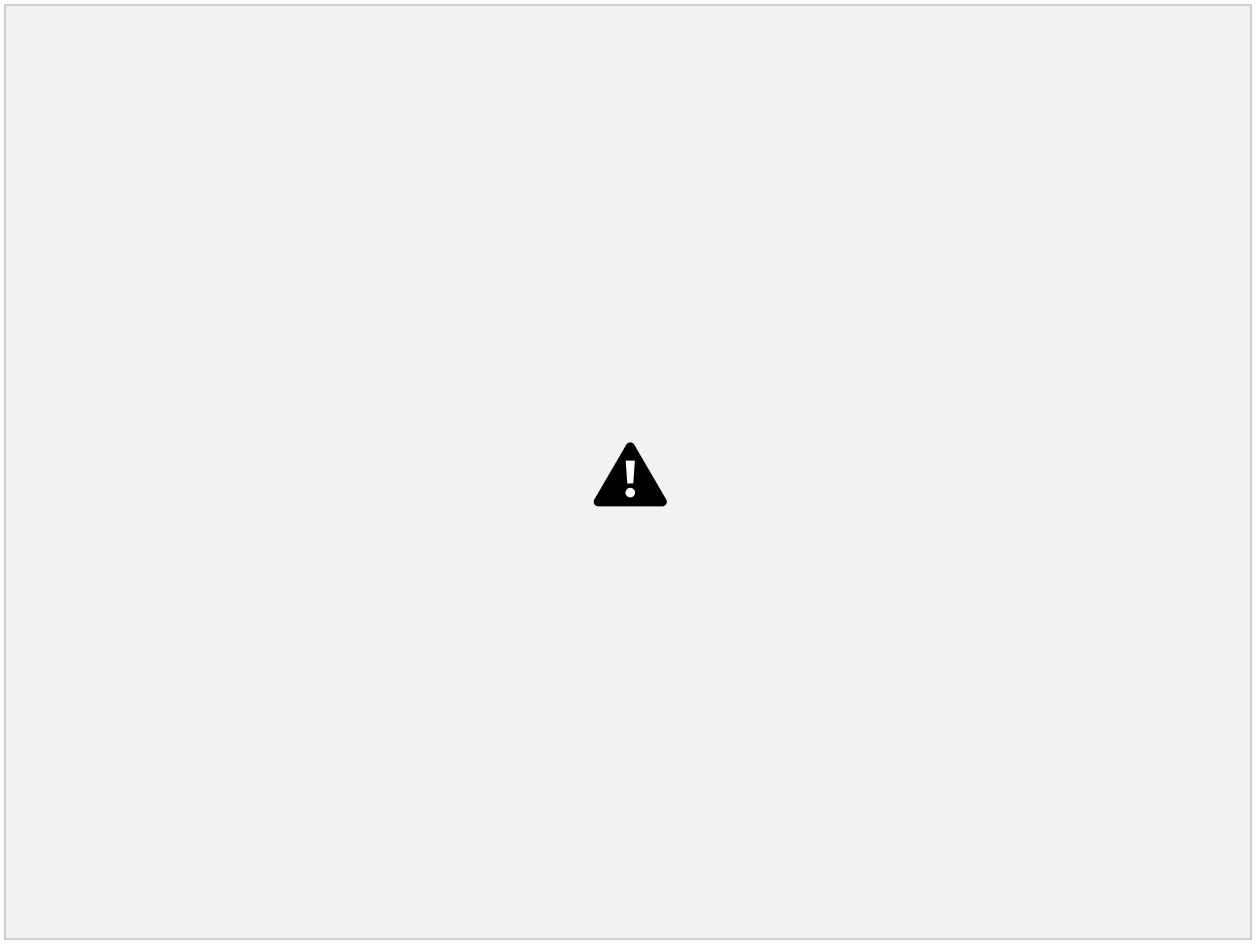


Can you regenerate this diagram based on the scope change?



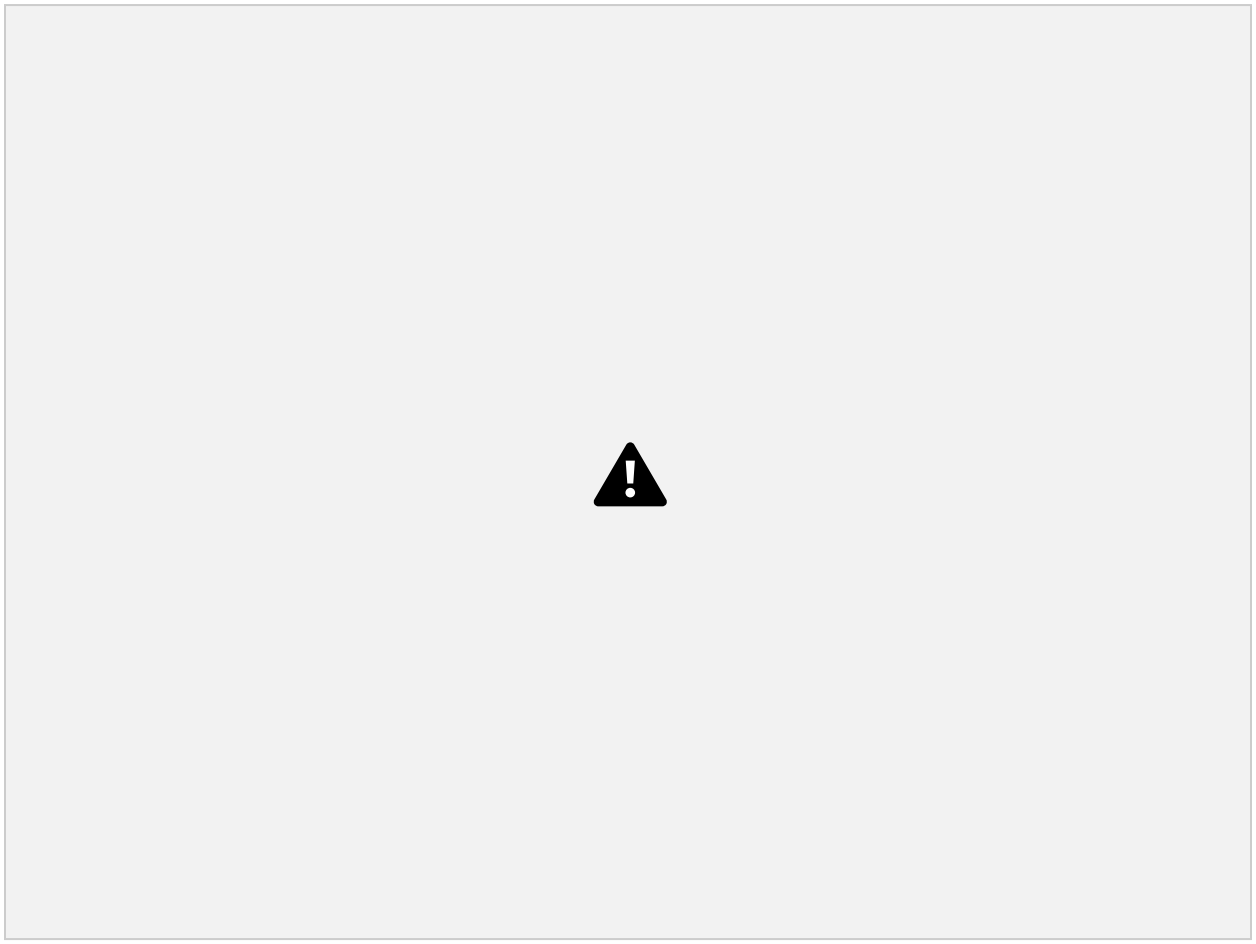
Can you regenerate this diagram based on the scope change?

Edit



Can you update this too and regenerate the image based on the new scope

Edit



Can you please regenerate this too because of the change in scope? As per mom there is no vc and all

Edit

Revised WP5.3 Deliverables, Tasks & Approximate Timeline

(Aligned with Updated Scope — No Full VC / No Closed-Loop Autonomous Optimization)

D0 — Requirement Gathering & Industrial Alignment

Duration: ~1.5 Months

Objective

Define industrial scope, anomaly monitoring expectations, KPI requirements, and alignment with WP5.1/mTorres/OMTC.

D0.1 Stakeholder Requirement Collection

Duration: ~2 Weeks

Tasks

- Conduct meetings with OMTC, WP5.1, mTorres.
- Identify industrial pain points.
- Define expected monitoring capabilities.
- Identify anomaly use cases.
- Understand expected operator workflow.

Outputs

- Requirement specification
- Use-case definition

D0.2 AFP/Laying Process Understanding

Duration: ~2 Weeks

Tasks

- Study AFP process stages:
    - approach
    - laydown
    - coursing
    - cut/add
    - retract
- Identify process variables and dependencies.
- Understand KPP/KPI definitions.

Outputs

- Process-state definition
- Process variable mapping

D0.3 Scope Alignment & Validation

Duration: ~2 Weeks

Tasks

- Align scope boundaries between WP5.1 and WP5.3.
- Clarify:
    - behavioral process twin
    - anomaly monitoring
    - offline analysis framework
- Remove non-scope items:
    - full physics twin
    - heavy VC

    - autonomous optimization

Outputs

- Approved scope baseline

D1 — System Architecture & Interface Definition

Duration: ~2 Months

D1.1 High-Level System Architecture

Duration: ~2 Weeks

Tasks

- Define overall architecture.
- Define:
    - ingestion flow
    - behavioral modeling flow
    - anomaly monitoring flow
    - visualization flow
- Define runtime vs offline workflows.

Outputs

- System architecture diagrams

D1.2 Functional Architecture Definition

Duration: ~3 Weeks

Tasks

Define major modules:

- ingestion
- behavioral modeling
- anomaly engine
- KPI framework
- analysis framework
- monitoring outputs

Outputs

- Functional architecture document

D1.3 Data Interface & Schema Definition

Duration: ~2 Weeks

Tasks

- Define interfaces with WP5.1.
- Define:
    - KPI schema
    - metadata structure
    - defect schema
    - process-state schema

Outputs

- ICD/interface document

D1.4 Monitoring & HMI Integration Definition

Duration: ~2 Weeks

Tasks

- Define dashboard outputs.
- Define anomaly visualization logic.
- Define operator alerts.

- Define expected process insight outputs.

Outputs

- HMI integration specification

D2 — Sample Data Collection & Modeling Preparation

Duration: ~2 Months

D2.1 Sample/Historical Data Collection

Duration: ~2 Weeks

Tasks

- Collect sample/historical mTorres datasets.
- Organize process datasets.
- Organize machine datasets.

Outputs

- Sample dataset repository

D2.2 KPI & SI Variable Definition

Duration: ~2 Weeks

Tasks

Define:

- process inputs
- process outputs

- prediction targets
- anomaly indicators
- KPI relationships

Outputs

- SI variable mapping

D2.3 Process-State & Label Definition

Duration: ~2 Weeks

Tasks

Define:

- process states
- operational phases
- defect labels
- event labels

Outputs

- Process labeling framework

D2.4 Modeling Dataset Structuring

Duration: ~2 Weeks

Tasks

- Create SI-ready datasets.
- Organize:
    - train
    - validation
    - test datasets
- Create time-window datasets.

Outputs

- Structured modeling datasets

D3 — Data Ingestion Pipeline Development

Duration: ~2 Months

D3.1 Ingestion Architecture Definition

Duration: ~2 Weeks

Tasks

- Define ingestion workflow.
- Define storage strategy.
- Define metadata handling.

Outputs

- Ingestion architecture

D3.2 Ingestion Pipeline Development

Duration: ~4 Weeks

Tasks

Develop:

- ingestion APIs
- parsers
- validation logic
- storage handling

Outputs

- Reusable ingestion framework

D3.3 Data Mapping for Behavioral Modeling

Duration: ~2 Weeks

Tasks

Map:

- KPIs
- defects
- process states
- contextual data
  to modeling-ready structures.

Outputs

- Modeling-ready mapped datasets

D3.4 Mock Data Integration Testing

Duration: ~2 Weeks

Tasks

- Validate ingestion with sample data.
- Validate schema compatibility.
- Validate future WP5.1 compatibility.

Outputs

- Validated ingestion pipeline

D4 — Actual Laying Data Integration & Dataset Preparation

Duration: ~2.5 Months

D4.1 Actual Laying Data Collection

Duration: ~2 Weeks

Tasks

- Receive real laying data from WP5.1.
- Import through ingestion pipeline.
- Validate dataset completeness.

Outputs

- Integrated production datasets

D4.2 SI Variable & KPI Refinement

Duration: ~2 Weeks

Tasks

Refine:

- KPIs
- anomaly indicators
- prediction targets
- process-state definitions

using actual laying data.

Outputs

- Refined SI variable set

D4.3 Production Dataset Preparation

Duration: ~3 Weeks

Tasks

Prepare:

- runtime-ready datasets
- process-window datasets
- sequence datasets
- scenario datasets

Outputs

- Production-ready SI datasets

D4.4 Defect & Process Correlation Mapping

Duration: ~2 Weeks

Tasks

- Align defects with process conditions.
- Identify anomaly signatures.
- Map process instability regions.

Outputs

- Defect-process mapping

D4.5 Dataset Validation & Coverage Analysis

Duration: ~2 Weeks

Tasks

- Validate KPI coverage.
- Validate process-state coverage.
- Validate modeling readiness.

Outputs

- Dataset validation report

D5 — Behavioral Modeling & System Identification

Duration: ~3 Months

D5.1 Process Input/Output Mapping

Duration: ~2 Weeks

Tasks

Define:

- expected process behavior
- model inputs
- model outputs
- prediction targets

Outputs

- Behavioral mapping definition

D5.2 SI Framework Setup

Duration: ~2 Weeks

Tasks

Setup:

- MATLAB/Python SI environment
- training workflows
- evaluation workflows

Outputs

- SI development environment

D5.3 Behavioral Model Development

Duration: ~5 Weeks

Tasks

Develop:

- time-series models
- lightweight ML models
- behavioral process models

Outputs

- Behavioral process models

D5.4 Offline Validation (Validation Loop 1)

Duration: ~3 Weeks

Tasks

Perform:

- train/test validation
- residual analysis
- error computation
- model comparison

Outputs

- Offline validation report

D5.5 Runtime Drift & Anomaly Logic

Duration: ~2 Weeks

Tasks

Define:

- anomaly thresholds
- expected process envelopes
- runtime drift indicators

Outputs

- Runtime anomaly framework

D6 — Behavioral Process Twin & Offline Analysis Framework

Duration: ~3 Months

D6.1 Behavioral Process Twin Integration

Duration: ~3 Weeks

Tasks

Integrate:

- SI models
- prediction workflows

- expected behavior generation

Outputs

- Behavioral process twin

D6.2 Process Trend Prediction Framework

Duration: ~3 Weeks

Tasks

Predict:

- KPI trends
- expected signal behavior
- anomaly likelihood
- process envelopes

Outputs

- Prediction framework

D6.3 KPI Correlation Framework

Duration: ~2 Weeks

Tasks

Analyze:

- KPI interactions
- process dependencies
- parameter influence

Outputs

- KPI correlation framework

D6.4 Root Cause Analysis Framework

Duration: ~2 Weeks

Tasks

Develop offline engineering analysis framework for:

- defect relationships
- process instability analysis
- anomaly investigation

Outputs

- RCA framework

D6.5 Sensitivity Analysis Framework

Duration: ~2 Weeks

Tasks

Analyze:

- parameter sensitivity
- dominant process variables
- process influence

Outputs

- Sensitivity analysis framework

D6.6 What-If Simulation Capability

Duration: ~2 Weeks

Tasks

Simulate:

- process parameter variations
- expected KPI changes
- anomaly behavior

Outputs

- What-if analysis framework

D7 — Anomaly Detection & Monitoring Intelligence

Duration: ~2 Months

D7.1 Runtime Anomaly Detection Framework

Duration: ~3 Weeks

Tasks

Develop:

- deviation detection
- anomaly scoring
- process envelope monitoring

Outputs

- Runtime anomaly framework

D7.2 Process Stability Monitoring

Duration: ~2 Weeks

Tasks

Develop:

- process stability indicators
- health indicators
- drift monitoring

Outputs

- Stability monitoring framework

D7.3 Early Warning & Alert Logic

Duration: ~2 Weeks

Tasks

Develop:

- warning thresholds
- alert conditions
- notification logic

Outputs

- Alert framework

D7.4 Runtime Validation Loop (Validation Loop 2)

Duration: ~2 Weeks

Tasks

Compare:

- predicted vs actual behavior

- runtime error metrics
- model drift

Outputs

- Runtime validation framework

D8 — Visualization & WP5.1 Integration

Duration: ~2 Months

D8.1 Prediction Output Interface

Duration: ~2 Weeks

Tasks

Define:

- prediction outputs
- metadata schema
- KPI output structures

Outputs

- Prediction output interface

D8.2 WP5.1 Visualization Integration

Duration: ~3 Weeks

Tasks

Integrate:

- predicted KPIs

- anomaly indicators
- process insights
  into WP5.1 visualization systems.

Outputs

- Visualization integration

D8.3 Dashboard & Monitoring Outputs

Duration: ~2 Weeks

Tasks

Provide:

- trend data
- anomaly indicators
- process envelopes
- operator insights

Outputs

- Dashboard-ready outputs

D8.4 Model Health & Confidence Indicators

Duration: ~2 Weeks

Tasks

Compute:

- model confidence
- prediction confidence
- runtime health metrics

Outputs

- Model-health framework

D9 — Continuous Learning & Model Improvement

Duration: ~1.5 Months

D9.1 Runtime Error Monitoring

Duration: ~2 Weeks

Tasks

Monitor:

- runtime prediction errors
- drift metrics
- anomaly false positives

Outputs

- Runtime error monitoring

D9.2 Incremental Model Improvement Workflow

Duration: ~2 Weeks

Tasks

Develop workflows for:

- model updates
- retraining
- recalibration

Outputs

- Model refinement workflow

D9.3 Continuous Learning Framework

Duration: ~2 Weeks

Tasks

Develop:

- runtime learning logic
- data update strategy
- improvement workflows

Outputs

- Continuous learning framework

In this this process anomaly detection, monitoring kpi changes, variations etc will be part of behaviour model (identified model?

Yes — exactly.
Based on the updated scope, anomaly detection and KPI variation monitoring are now becoming a major responsibility of the **behavioral identified model**.

But there is an important architectural distinction.

Correct understanding now

The identified behavioral model itself is NOT directly:

"anomaly detector"

Instead:

It learns NORMAL process behavior.

Then anomaly detection is performed by:

Comparing:
Expected behavior (from identified model)
vs
Actual runtime behavior

So architecture becomes

Actual Process Data
↓
System Identification
(learn normal behavior)
↓
Behavioral Process Model
↓
Predict Expected Process Behavior
↓
Compare with Actual Runtime Data
↓
Deviation / Drift / KPI Variation
↓
Anomaly Detection

Example

Suppose identified model learned:

Speed = 200
Temperature = 72
Tow Pressure should remain 50 ± 3

Runtime

Actual machine:

Tow Pressure = 39

Then:

Deviation = High
→ Process anomaly

So YES:

These become part of the behavioral model workflow:

- KPI trend prediction
- expected process envelope
- process behavior prediction
- runtime deviation monitoring
- process drift detection
- anomaly scoring
- process stability monitoring

Important distinction

The SI/behavioral model gives:

Expected process behavior

The anomaly engine computes:

Difference between:
Expected vs Actual

So your architecture should now look like this

1. System Identification

Learns:

Normal process behavior

Outputs:

- behavioral model
- expected KPI relations
- expected signal trends

---

2. Behavioral Process Twin

Uses identified model to generate:

Expected process outputs
Expected KPI trends
Expected process envelopes

---

3. Runtime Monitoring & Anomaly Detection

Compares:

Actual runtime data
vs
Predicted/expected behavior

Computes:

- deviation
- drift
- anomaly score
- KPI instability

So anomaly detection is now tightly coupled with:

behavioral model

AND

runtime comparison logic

Very important scope clarification

This is NOT classical anomaly detection like:

unsupervised AI anomaly detection only

Instead it is:

Model-based anomaly detection

Meaning:

Expected behavior is generated by the identified behavioral model.

Then:

Deviation from expected behavior
=
anomaly

So your D5/D6/D7 relationship now becomes

Deliverable	Role
D5	Learn behavioral process model
D6	Generate expected process behavior/trends

D7	Compare runtime behavior vs expected behavior

Runtime monitoring now mainly checks

KPI deviations

Example:

- pressure instability
- abnormal tension
- unusual vibration
- temperature drift

Process-state abnormalities

Example:

- laydown phase taking too long
- unexpected process transition
- unstable process response

Signal behavior anomalies

Example:

- oscillations
- spikes
- drift
- unexpected trend changes

This aligns VERY WELL with the MOM

Especially these points:

> “Identification of anomalies in live data stream”

and

> “variations in pressure or tension”

This is exactly:

behavioral-model-based monitoring

NOT:

- full AI autonomy
- deep optimization control
- advanced reinforcement learning

Final simplified understanding

System Identification

Learn how process normally behaves

Behavioral Process Twin

Predict how process SHOULD behave

Runtime Monitoring

Check whether actual process deviates from expected behavior

Anomaly Detection

Deviation beyond acceptable threshold

That is now the core of your updated WP5.3 scope.

runtime learning logic Is this required?. Because you mentioned previously that they need only offline

Yes — good catch.
Based on the updated MOM/scope, a full **runtime learning logic** is probably too strong and too research-oriented.

You should reduce it significantly.

What they MOST LIKELY want now

NOT this:

Online self-learning AI system
that continuously retrains itself automatically during production

That would be:

- risky industrially
- difficult to validate
- difficult to certify
- not aligned with MOM

What they likely want instead

Runtime validation & monitoring

Meaning:

Monitor:
Predicted behavior
vs
Actual behavior

and then:

Offline engineering review
→ model improvement later

So the correct flow now becomes

Runtime (online)

Actual process data
→ compare with expected behavior
→ detect anomalies
→ compute deviations
→ generate alerts
→ visualize KPIs

ONLY.

Then Offline

Later:

Collect runtime deviations
→ engineer analyzes them
→ retrain/refine model offline
→ redeploy updated model

This is MUCH more realistic industrially.

So remove this wording

❌ Remove:

- runtime learning
- online learning
- self-learning system
- autonomous model adaptation
- continuous runtime retraining

Replace with

✅ Better wording:

- Runtime validation framework
- Model performance monitoring
- Drift monitoring
- Offline model refinement workflow
- Model recalibration workflow
- Runtime deviation monitoring

So D9 should probably become

D9 — Runtime Validation & Model Improvement Framework

NOT:

Continuous Learning Framework

Revised D9

D9 — Runtime Validation & Model Improvement Framework

Duration: ~1 Month

D9.1 Runtime Prediction Validation

Duration: ~2 Weeks

Tasks

- Compare predicted vs actual behavior.
- Compute runtime error metrics.
- Monitor KPI deviations.
- Detect model drift.

Outputs

- Runtime validation framework

D9.2 Offline Model Refinement Workflow

Duration: ~2 Weeks

Tasks

Define workflow for:

- collecting runtime deviations
- offline engineering review
- retraining/recalibration process
- updated model deployment

Outputs

- Model improvement workflow

VERY IMPORTANT difference

Before (too advanced)

Runtime:

Data → retrain model automatically

Now (industrial realistic)

Runtime:
Monitor only

Offline:
Improve model later

This aligns MUCH better with these MOM statements

> “root cause analysis is post-process activity”

and

> “details and analysis can be performed as a second step”

and

> “industrial deployment focus”

Final recommendation

You should now avoid these terms in presentations:

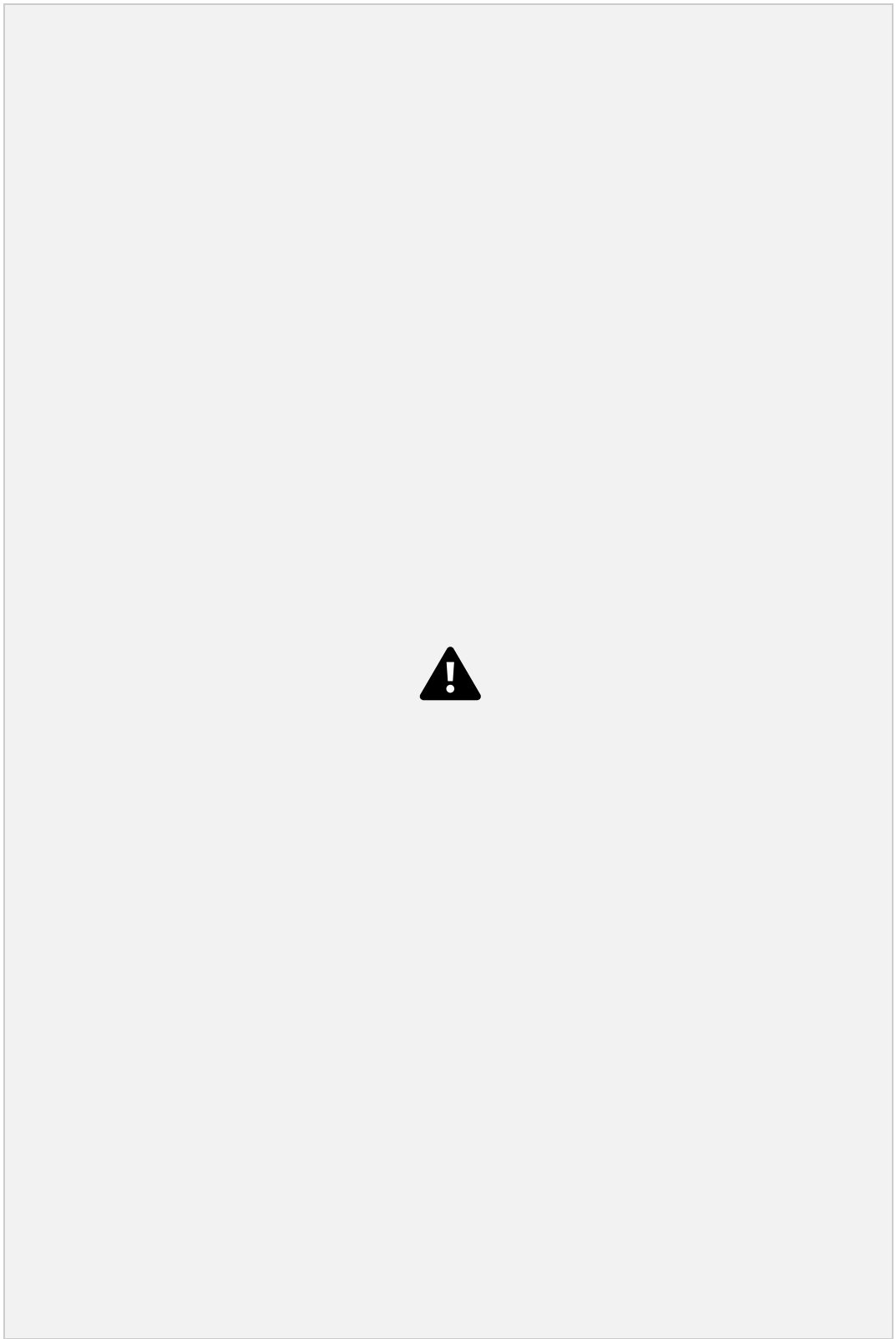
❌ Avoid:

- autonomous learning
- self-learning AI
- online retraining
- adaptive optimization
- closed-loop autonomous AI

✅ Use:

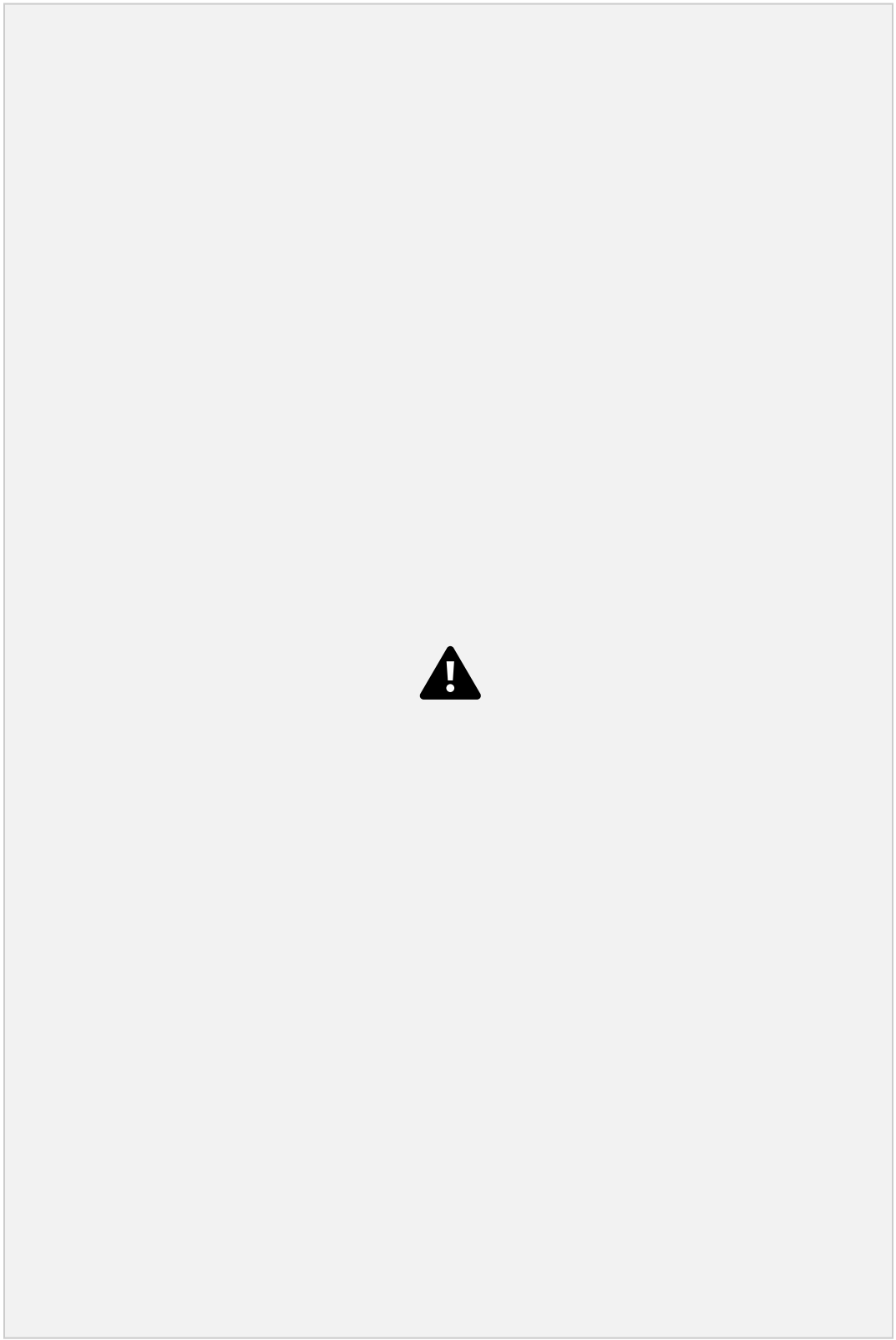
- runtime validation
- deviation monitoring
- anomaly monitoring
- offline model refinement
- engineering feedback workflow
- model recalibration

This will align MUCH better with the actual industrial expectation.



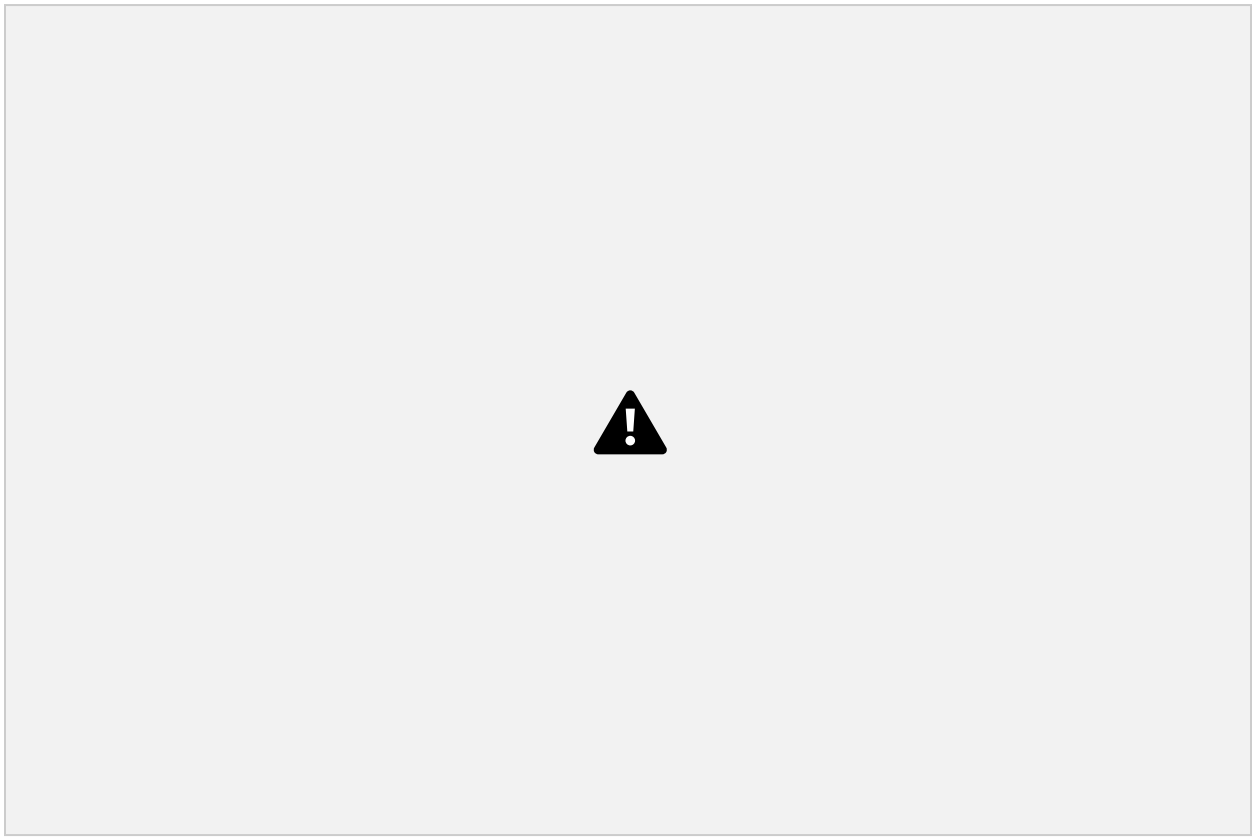
can you regenearte the diagram with deliverables we discuss don't mention engineering analysis part Time series framework & Process Anomlay detection emorical behavioral model

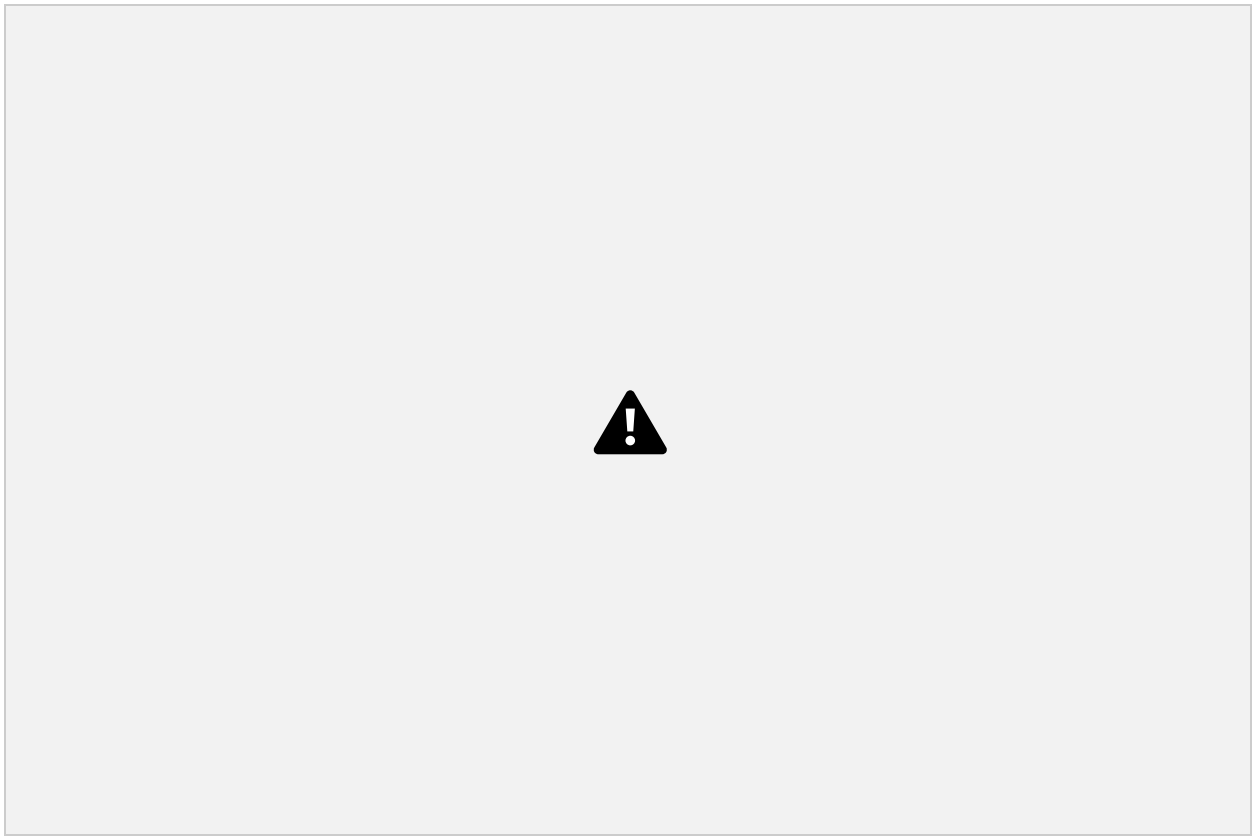
(or system identuification) HMI interation for visualization (process acriation, process anomaly dtecetion etc)



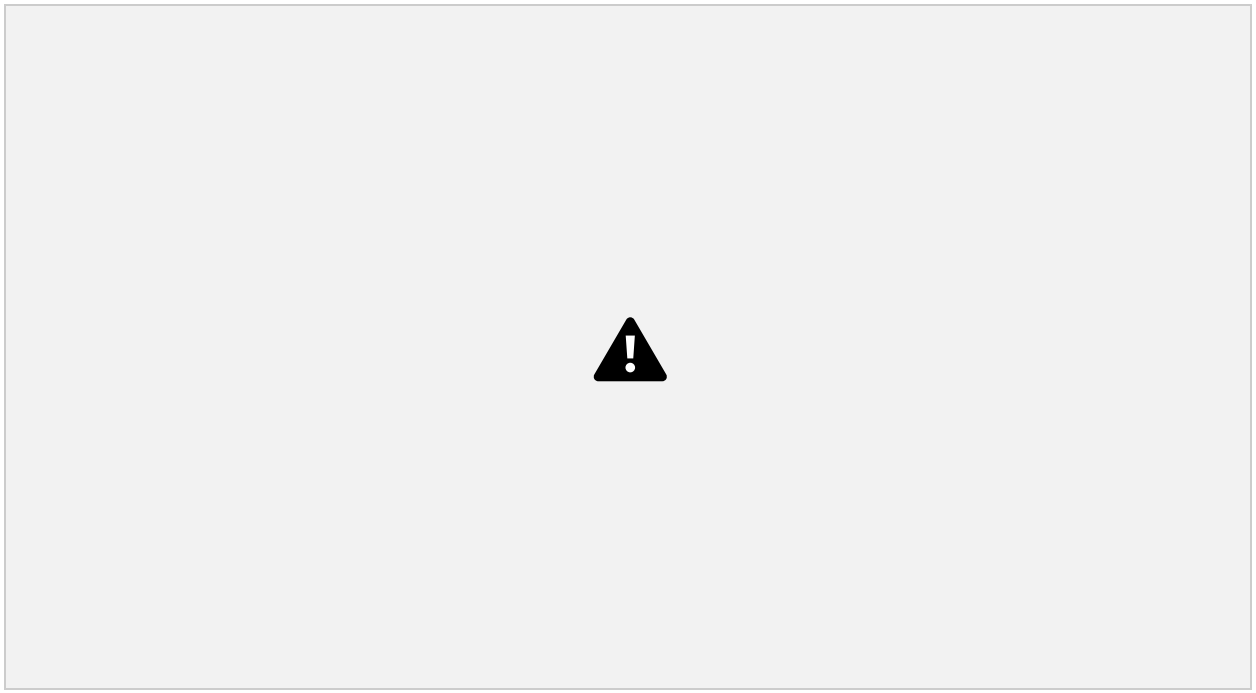
Edit

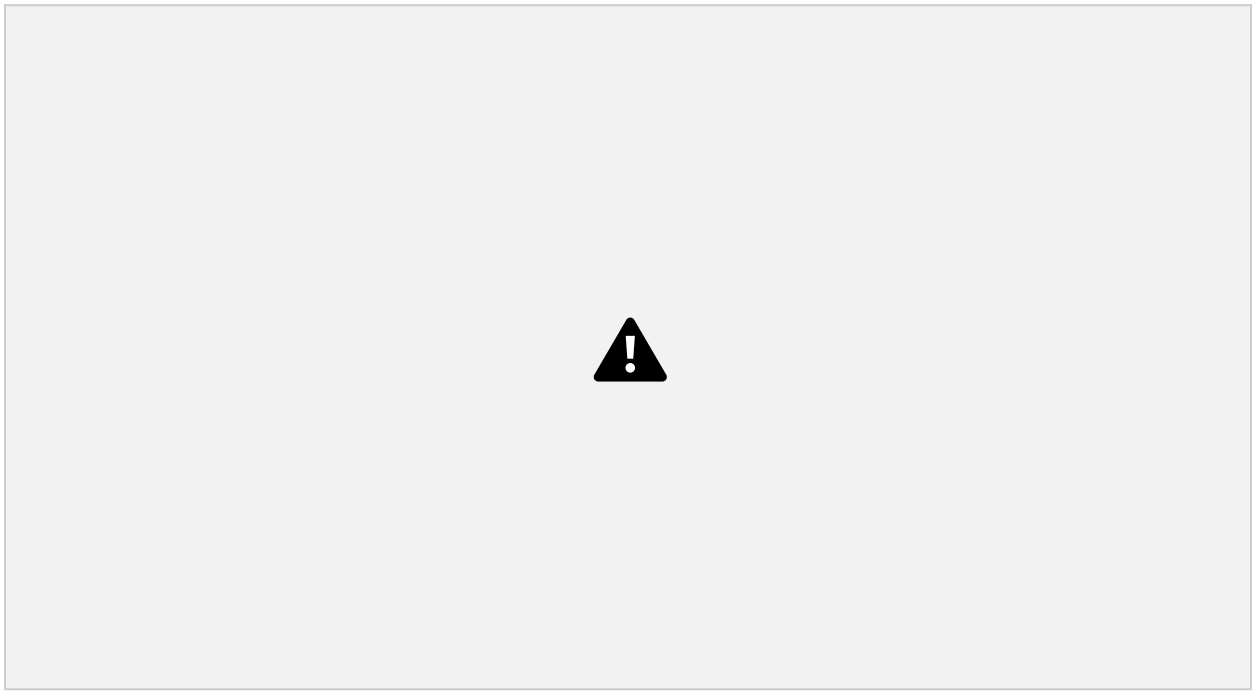
Edit



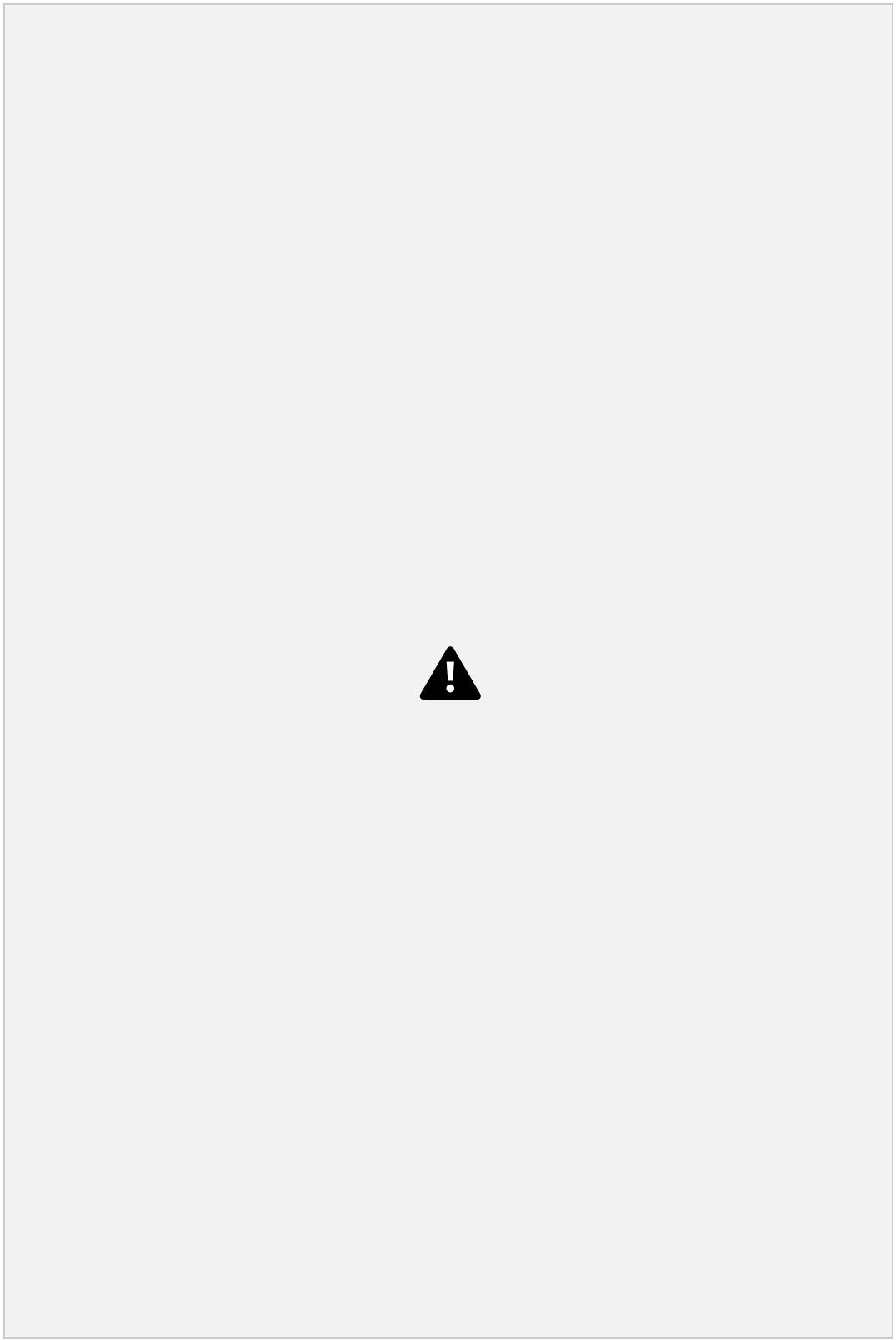


Edit





Edit

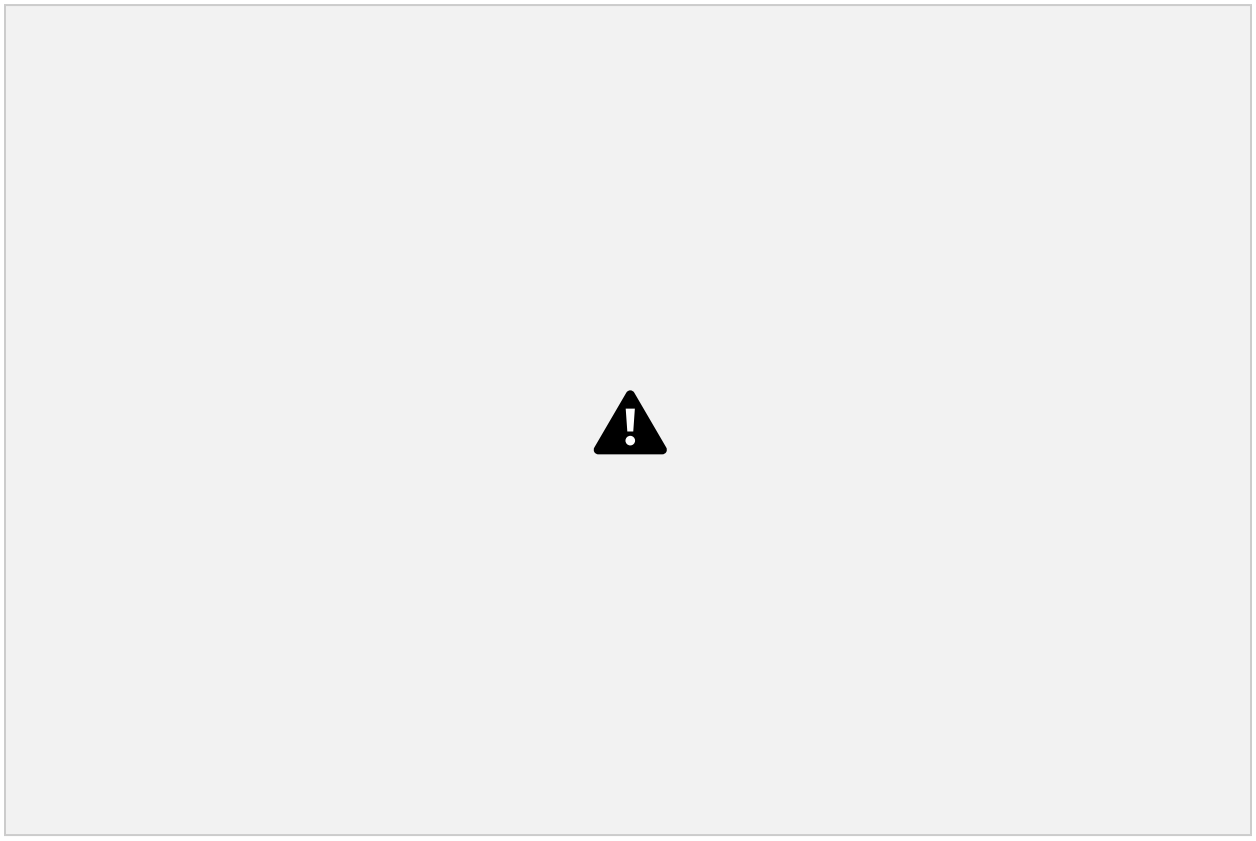


please use this template

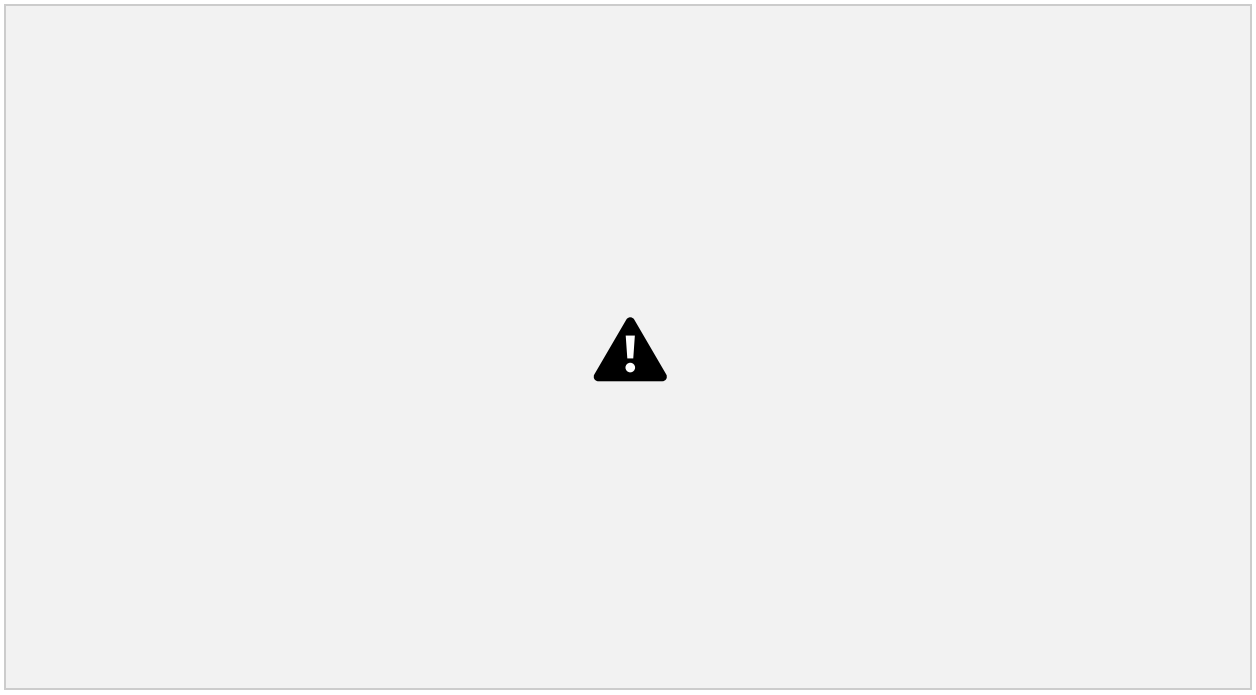
Edit

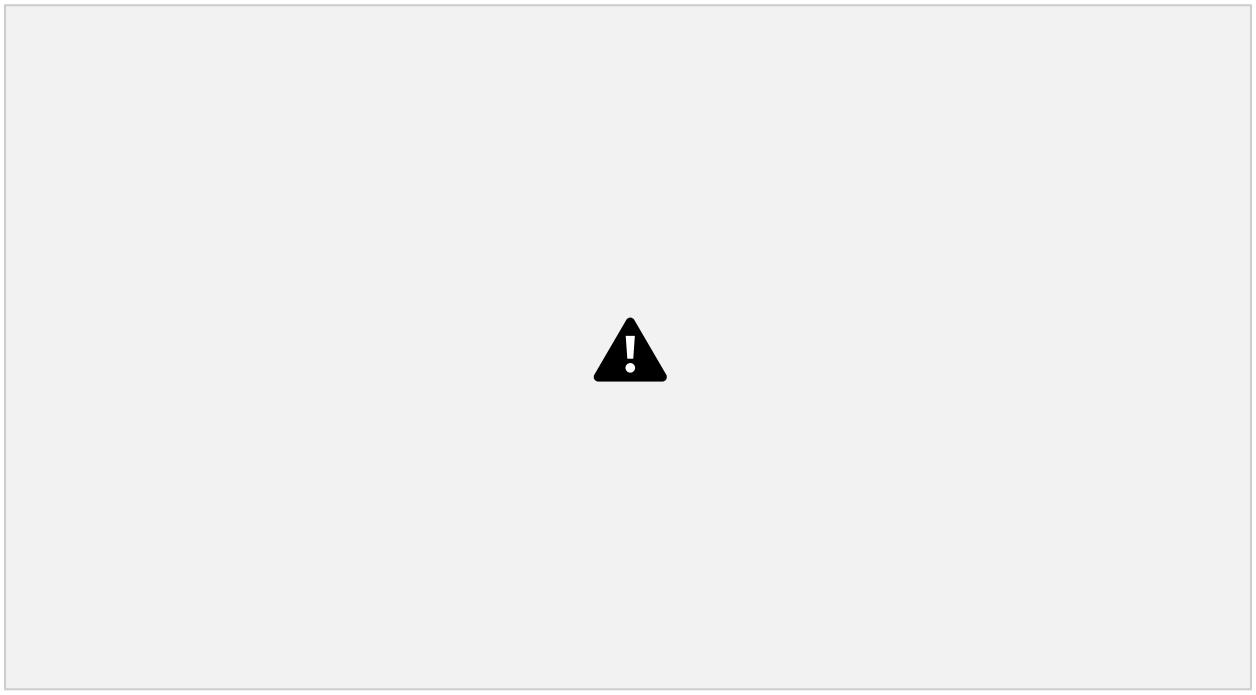
Edit

Edit



Can you regenerate this diagram based on our discussion and new scope

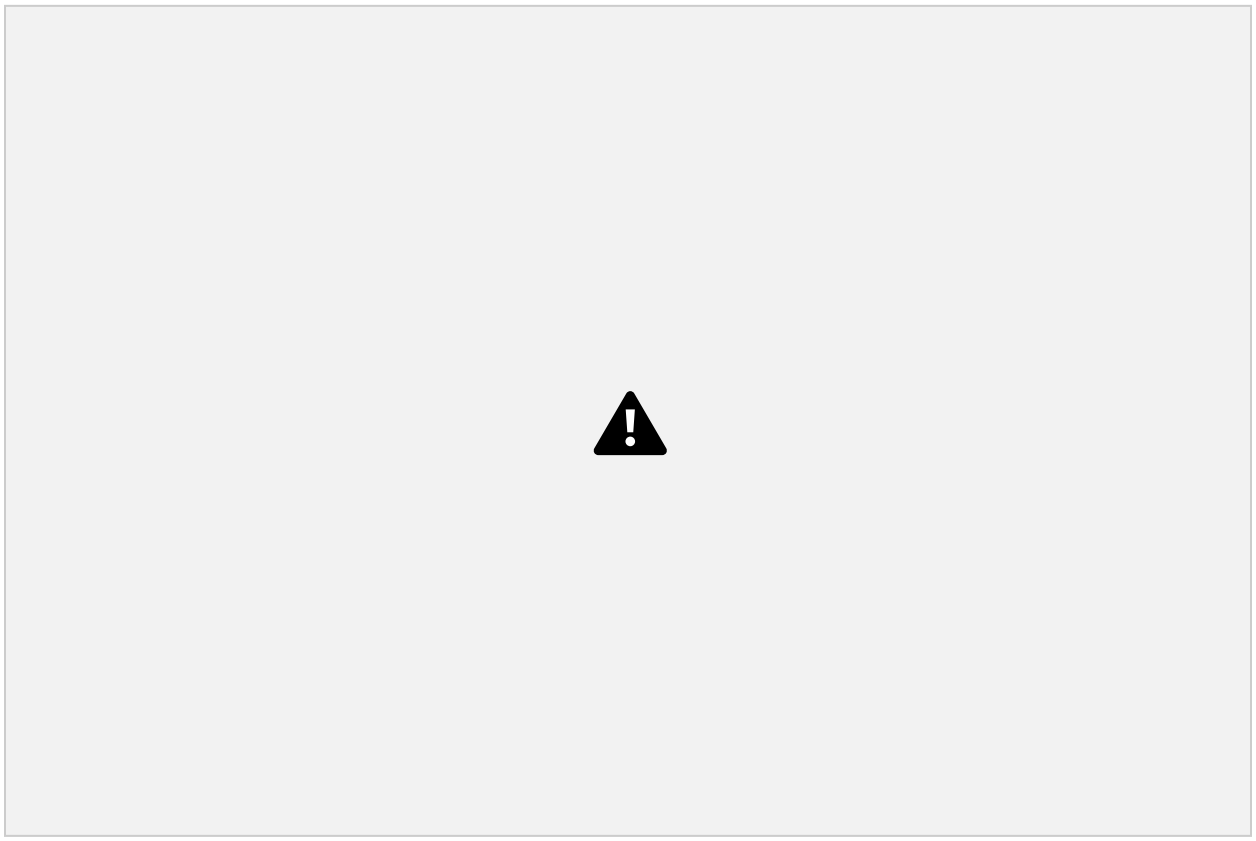


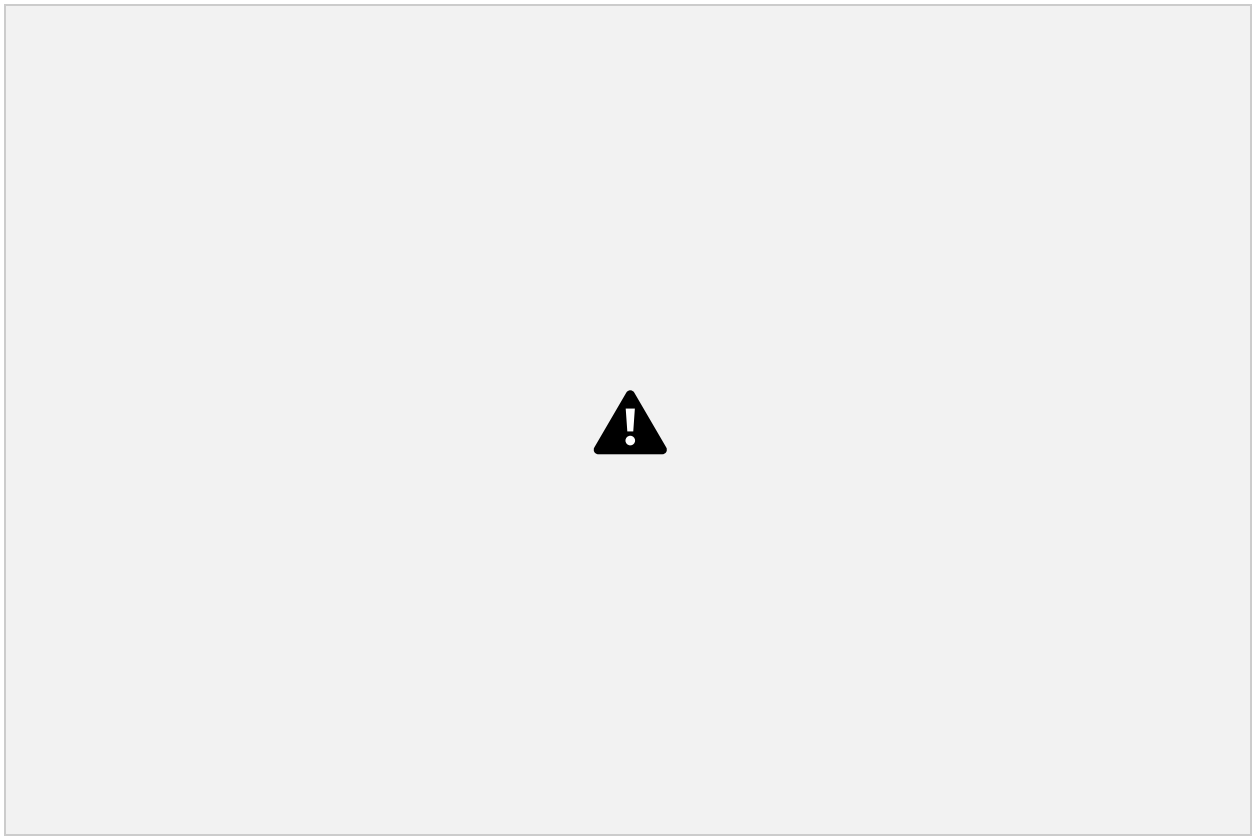


Edit

Edit

can you create a high level diagram from teh above image with lesser text





Edit
can you once again give all the list of deliverables and detailed list of task and days required to complete the tasks

WP5.3.1 – Process Intelligence & Optimization

Deliverables, Detailed Tasks & Estimated Duration

(Updated based on latest MoMs and revised scope)

D1 — System Architecture & Scope Definition

Objective

Define the overall WP5.3.1 architecture, interfaces with WP5.1, modeling scope, anomaly detection scope, and HMI visualization concept.

Expected Output

- High-level architecture
- Data flow definition
- Scope boundaries between WP5.1 and WP5.3
- Initial process intelligence workflow

Estimated Duration

~4 Weeks

Tasks

1.1 Requirement Understanding & MoM Consolidation

Review discussions with OMTC, mTorres, WP5.1, and internal teams to understand:

- expected process intelligence scope,
- anomaly monitoring expectations,
- visualization requirements,
- process behavior understanding needs.

Duration

~4 Days

1.2 Define WP5.1 ↔ WP5.3 Responsibilities

Clearly define:

- WP5.1 responsibilities:
    - data acquisition,
    - storage,
    - preprocessing infrastructure,
    - monitoring infrastructure.
- WP5.3 responsibilities:
    - behavioral modeling,
    - anomaly analysis,
    - KPI intelligence,
    - HMI insight generation.

Duration

~3 Days

1.3 Define Overall Process Intelligence Architecture

Define:

- ingestion flow,
- modeling flow,
- anomaly framework,
- HMI integration flow,
- validation loop.

Duration

~5 Days

1.4 Define Data Categories & KPI Scope

Identify:

- process KPIs,
- machine parameters,
- contextual data,
- defect indicators,
- execution states.

Duration

~4 Days

1.5 Define Visualization & HMI Concept

Define:

- process activation visualization,
- anomaly overlays,
- KPI trends,
- expected vs actual visualization,
- 3D geometry superimposition concept.

Duration

~4 Days

1.6 Prepare Architecture Documentation & Review

Create:

- architecture diagrams,
- interface definitions,
- workflow diagrams,
- presentation material.

Duration

~4 Days

D2 — Sample Data Collection & Dataset Understanding (mTorres Data)

Objective

Use sample/historical mTorres data to start early development before actual laying data becomes available.

Expected Output

- Initial sample dataset
- KPI understanding
- Dataset structure understanding
- Early process signal analysis

Estimated Duration

~5 Weeks

Tasks

2.1 Coordinate Sample Dataset Access

Coordinate with mTorres and WP5.1 teams to obtain:

- historical datasets,
- sample machine signals,
- KPI information,
- defect/event samples.

Duration

~5 Days

2.2 Understand Dataset Structure & Signals

Study:

- timestamps,
- sampling rates,
- process phases,
- available KPIs,
- available contextual information.

Duration

~5 Days

2.3 Define Modeling Variables

Define:

- inputs,
- outputs,
- anomaly indicators,
- target KPIs,
- expected process behaviors.

Duration

~6 Days

2.4 Define Process States & Activation Logic

Identify:

- layup phases,
- travel states,
- cut/add states,
- process transitions,
- activation sequences.

Duration

~4 Days

2.5 Initial Dataset Exploration & Visualization

Perform:

- signal visualization,
- KPI trend exploration,
- anomaly example identification,
- process variation observation.

Duration

~5 Days

D3 — Data Ingestion & Modeling Dataset Preparation

Objective

Ingest WP5.1 datasets and prepare modeling-ready datasets for:

- time-series analysis,
- anomaly detection,
- empirical behavioral modeling.

Expected Output

- Modeling-ready datasets
- Structured time-series data
- Prepared SI datasets

Estimated Duration

~6 Weeks

Tasks

3.1 Implement Data Ingestion Pipeline

Create interfaces to ingest:

- KPIs,
- machine signals,
- defect data,
- contextual data,
- execution logs from WP5.1.

Duration

~8 Days

3.2 Dataset Alignment & Synchronization

Prepare synchronized datasets suitable for:

- anomaly analysis,
- behavioral modeling,
- KPI comparisons.

Duration

~5 Days

3.3 KPI Structuring & Aggregation

Organize:

- KPI groupings,
- parameter relationships,
- machine state mapping,
- process stage mapping.

Duration

~4 Days

3.4 Sequence & Window Generation

Prepare:

- time windows,
- rolling sequences,
- process activation intervals,
- event segmentation.

Duration

~5 Days

3.5 Dataset Structuring for SI & Time-Series Frameworks

Prepare datasets suitable for:

- ARX/state-space modeling,
- ML-based behavioral learning,
- anomaly frameworks.

Duration

~6 Days

3.6 Dataset Validation & Documentation

Validate:

- completeness,
- consistency,
- modeling suitability.

Duration

~4 Days

D4 — Time-Series Analysis & Process Anomaly Detection Framework

Objective

Develop anomaly monitoring and process variation detection framework.

Expected Output

- Time-series anomaly framework
- KPI deviation monitoring
- Process variation indicators

Estimated Duration

~8 Weeks

Tasks

4.1 Define Time-Series Monitoring Strategy

Define:

- monitoring logic,
- KPI tracking strategy,
- process variation indicators.

Duration

~5 Days

4.2 Implement Process Variation Detection

Detect:

- drifts,
- spikes,
- oscillations,
- KPI instability,
- abnormal process behavior.

Duration

~8 Days

4.3 Develop Anomaly Scoring Logic

Develop:

- severity indicators,
- anomaly scoring,

- warning thresholds.

Duration

~6 Days

4.4 Develop Process Health Indicators

Create:

- stability indicators,
- process quality indicators,
- runtime process condition metrics.

Duration

~5 Days

4.5 Validate Detection Framework with Sample Data

Validate anomaly framework using:

- sample datasets,
- known abnormal cases,
- process variations.

Duration

~6 Days

D5 — Empirical Behavioral Model (System Identification)

Objective

Learn expected process behavior using data-driven empirical modeling.

Expected Output

- Behavioral process model
- Expected KPI prediction
- Expected process envelopes

Estimated Duration

~10 Weeks

Tasks

5.1 Define Behavioral Modeling Strategy

Select:

- SI approaches,
- empirical model structure,
- ML/SI framework.

Duration

~5 Days

5.2 Define Input-Output Relationships

Map:

- machine/process parameters,
- KPI dependencies,
- contextual influences.

Duration

~6 Days

5.3 Develop Initial Behavioral Models

Develop:

- ARX/state-space models,
- regression models,
- empirical behavioral models.

Duration

~10 Days

5.4 Train & Validate Behavioral Models

Perform:

- model training,
- parameter estimation,
- prediction validation.

Duration

~8 Days

5.5 Compare Expected vs Actual Behavior

Validate:

- predicted KPIs,
- deviations,
- process envelopes,
- residuals.

Duration

~6 Days

5.6 Refine Behavioral Model

Improve:

- prediction quality,

- stability,
- process representation.

Duration

~5 Days

D6 — HMI Integration & Visualization Framework

Objective

Integrate insights, anomalies, and behavioral outputs into HMI visualization.

Expected Output

- HMI visualization framework
- KPI dashboards
- Anomaly overlays on 3D geometries

Estimated Duration

~7 Weeks

Tasks

6.1 Define Visualization Requirements

Define:

- dashboard structure,
- operator views,
- KPI visualizations,
- anomaly overlays.

Duration

~5 Days

6.2 Define HMI Data Exchange Interface

Define interface between:

- WP5.3 outputs,
- WP5.1 HMI/dashboard system.

Duration

~4 Days

6.3 Implement KPI & Trend Visualization

Visualize:

- KPI trends,
- expected vs actual,
- process state indicators.

Duration

~7 Days

6.4 Implement Process Activation Visualization

Visualize:

- process phases,
- process transitions,
- activation timelines.

Duration

~5 Days

6.5 Implement Anomaly Visualization on 3D Geometry

Superimpose:

- anomalies,
- deviations,
- KPI severity
  onto part geometries.

Duration

~8 Days

6.6 Validate Visualization Workflow

Validate:

- usability,
- correctness,
- visualization clarity.

Duration

~4 Days

D7 — Validation & Model Refinement Loop

Objective

Validate models and improve anomaly framework iteratively.

Expected Output

- Refined anomaly framework
- Improved behavioral model
- Validated KPI prediction logic

Estimated Duration

~5 Weeks

Tasks

7.1 Compare Predicted vs Actual Behavior

Analyze:

- KPI deviations,
- prediction errors,
- anomaly behavior.

Duration

~5 Days

7.2 Analyze Residuals & Detection Accuracy

Evaluate:

- anomaly detection quality,
- false positives,
- missed detections.

Duration

~5 Days

7.3 Refine Detection Logic & Thresholds

Improve:

- anomaly thresholds,
- severity logic,
- process condition classification.

Duration

~5 Days

7.4 Refine Behavioral Model Parameters

Improve:

- model robustness,
- KPI prediction quality,
- process representation.

Duration

~5 Days

7.5 Final Validation & Demonstration

Perform:

- end-to-end validation,
- demonstration preparation,
- stakeholder review.

Duration

~5 Days